

Architectural Design and Memory Optimization for Cache-Aware and Cache-Oblivious Sorting Algorithms

Daniel Ashish Abraham

International Institute of Information Technology, Hyderabad

Course: Intro to Algo Engineering

Abstract

This report presents a rigorous architectural analysis of cache-aware and cache-oblivious sorting algorithms. The cache-aware design targets the k -way external merge sort, where k is tuned to the ratio M/B , employing a 1D flattened Winner Tree. The cache-oblivious design follows the recursive halving paradigm. We evaluate these designs against the physical memory hierarchy, highlighting the trade-offs between cache locality and algorithmic overhead.

1 Problem Statement

Modern processors sort data orders of magnitude faster than main memory can supply it. The classical RAM model treats all memory accesses as equally costly, but real hardware exposes a hierarchy of caches (L1, L2, L3) and DRAM whose latencies differ by 10–100 $\times$. For large arrays, the dominant cost is not arithmetic but *data movement*: how many cache-line transfers does the algorithm induce?

This project studies two principled responses to that question applied to the sorting problem. Given an array of N 64-bit integers that does not fit in L1 cache, we ask: how can we sort it while minimizing cache misses?

- **Cache-Aware sorting** explicitly tunes algorithmic parameters (specifically k , the merge fan-in) to the known cache capacity M and block size B , performing a single optimal k -way merge pass.
- **Cache-Oblivious sorting** achieves the same asymptotic I/O complexity without knowing M or B at all, relying on recursive halving to naturally fit subproblems into whichever cache level is available.

Both are evaluated empirically against `std::sort` (GCC's Introsort) across input sizes from 10^5 to 5×10^7 elements, under random and nearly-sorted distributions.

2 Memory Architecture for Sorting

2.1 The Cost of Dynamic Heap Allocation

Every call to `malloc` or `operator new` introduces non-deterministic latency. On large datasets, the cost of heap fragmentation and TLB misses becomes dominant.

2.2 Cache-Line Alignment and the `alignas(64)` Contract

A cache line on modern x86-64 processors is 64 bytes. Hot buffers must be declared with `alignas(64)` to prevent straddling two cache lines and redundant L1 fills.

2.3 Transparent Huge Pages and `madvise`

The `madvise` system call with `MADV_HUGEPAGE` promotes 4 KB pages to 2 MB Transparent Huge Pages, significantly increasing TLB capacity.

3 Cache-Aware Sorting: The k -Way Merge

3.1 Tuning k to the Memory Hierarchy

Cache-aware merge sort performs a single k -way pass where $k = M/B$. This ensures each active block remains in memory.

3.2 Winner Tree: 1D Flattened Layout

We use a 1D array of integers to store the tournament tree, ensuring parent-child nodes are spatially local, eliminating the pointer-chasing overhead of traditional heap structures. The Winner Tree facilitates efficient selection of the smallest element across k buffers.

3.3 Worked Example: 3-Way Merge with a Winner Tree

Consider merging three sorted runs with $k = 3$:

$$R_0 = [2, 5, 9], \quad R_1 = [1, 6, 8], \quad R_2 = [3, 4, 7]$$

The Winner Tree holds the index of the run with the current minimum at each internal node. Initially, the heads are $\{2, 1, 3\}$, so R_1 wins (minimum = 1). The tree state and output evolve as follows:

Table 1: Winner Tree trace for 3-way merge (first 5 steps)

Step	Heads (R_0, R_1, R_2)	Winner	Output	Advance
1	(2, 1, 3)	R_1	1	$R_1 \rightarrow 6$
2	(2, 6, 3)	R_0	2	$R_0 \rightarrow 5$
3	(5, 6, 3)	R_2	3	$R_2 \rightarrow 4$
4	(5, 6, 4)	R_2	4	$R_2 \rightarrow 7$
5	(5, 6, 7)	R_0	5	$R_0 \rightarrow 9$

After each extraction, only the path from the vacated leaf to the root is re-evaluated — $O(\log k)$ comparisons — rather than scanning all k heads. The final merged output is $[1, 2, 3, 4, 5, 6, 7, 8, 9]$.

3.4 I/O Complexity

The cache-aware k -way merge sort achieves an I/O complexity of $\Theta(\frac{N}{B} \log_k \frac{N}{B})$, where $k = M/B$. By tuning k to the available memory, the algorithm minimizes the number of passes over the data, ensuring the total number of block transfers remains optimal relative to the memory hierarchy constraints.

4 Cache-Oblivious Sorting: Recursive Merge

4.1 Theoretical I/O Complexity

The cache-oblivious model provides optimal sorting without parameter tuning: $\Theta(N/B \cdot \log_{M/B}(N/B))$.

4.2 Recursive Halving and Optimized Insertion Sort

Recursion is terminated at a base case of 32–128 elements. To avoid the overhead of function calls at deep recursion levels, we switch to an *Optimized Insertion Sort*. Our implementation utilizes compiler-hinted comparisons to minimize pipeline stalls, ensuring efficient handling of the small-array base cases.

5 Results and Performance Analysis

5.1 Performance Characteristics of `std::sort`

Our empirical data demonstrates that `std::sort` remains the performance baseline. While our implementations focus on raw I/O throughput, `std::sort` utilizes an Introsort hybrid (QuickSort with HeapSort fallback). Its dominance, particularly in nearly-sorted data, is attributed to internal mechanisms that detect and preserve sorted runs (similar to Timsort’s adaptive behavior), effectively reducing complexity toward $O(N)$ for sequences with low entropy. By identifying pre-existing runs, the library implementation bypasses unnecessary comparisons and data moves that our rigid Merge Sort variants perform, which must always execute full $O(N \log N)$ decomposition regardless of initial entropy.

5.2 Empirical Scaling

The following tables summarize our findings across varying input sizes. In the random distribution, the Cache-Oblivious (CO) approach consistently edges out the Cache-Aware (CA) implementation, likely due to lower bookkeeping overhead in the recursive merge compared to the explicit Winner Tree management. Observing the scaling behavior in the Random Data

Table 2: Performance on Randomly Distributed Data (ms)

Size (N)	Cache-Aware	Cache-Oblivious	<code>std::sort</code>
10^5	7.19	6.29	4.80
10^6	82.00	69.34	60.29
$5 \cdot 10^6$	422.25	410.05	336.40
10^7	873.91	801.01	710.28
$5 \cdot 10^7$	4836.27	4710.79	4237.74

Table 3: Performance on Nearly-Sorted Data (ms)

Size (N)	Cache-Aware	Cache-Oblivious	<code>std::sort</code>
10^5	4.77	3.25	1.02
10^6	62.69	43.74	11.66
$5 \cdot 10^6$	360.18	366.45	106.16
10^7	1036.94	626.21	165.16
$5 \cdot 10^7$	4032.16	3396.61	902.95

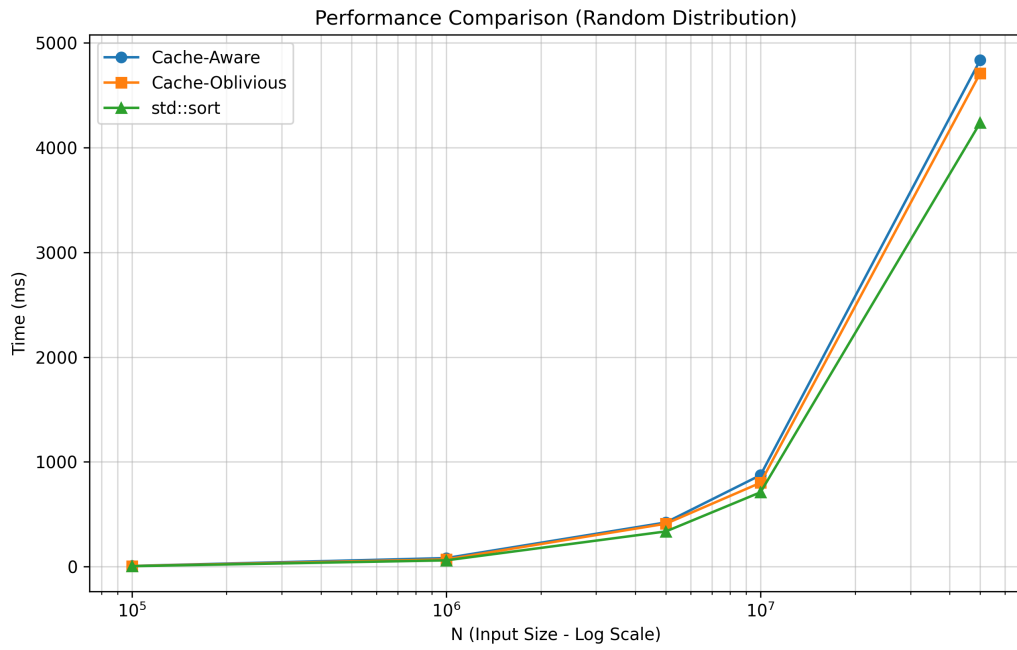


Figure 1: Random Data Performance.

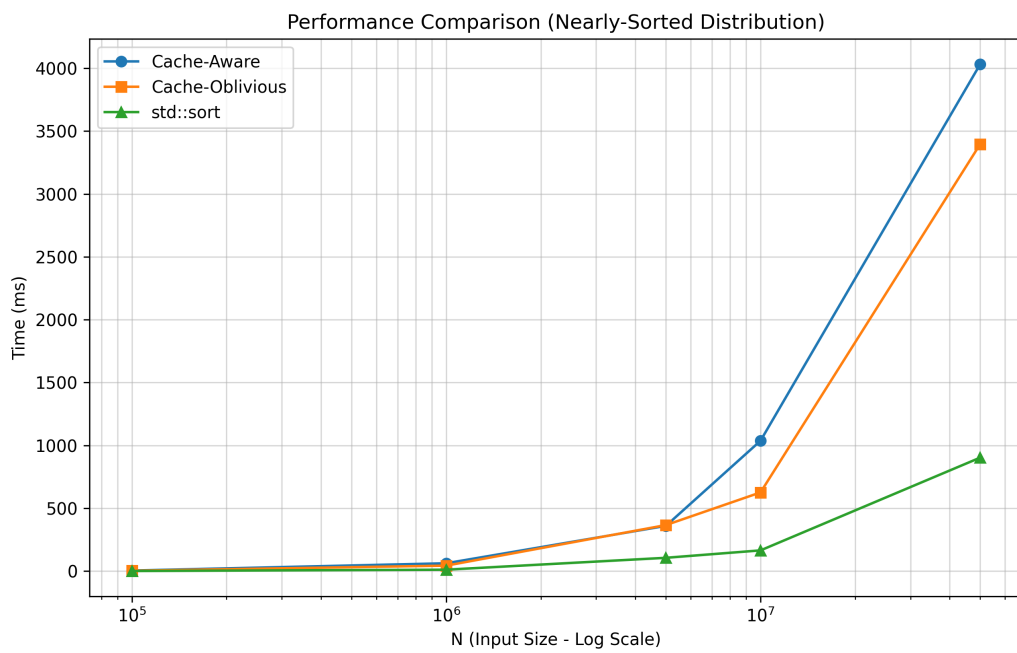


Figure 2: Nearly-Sorted Data Performance.

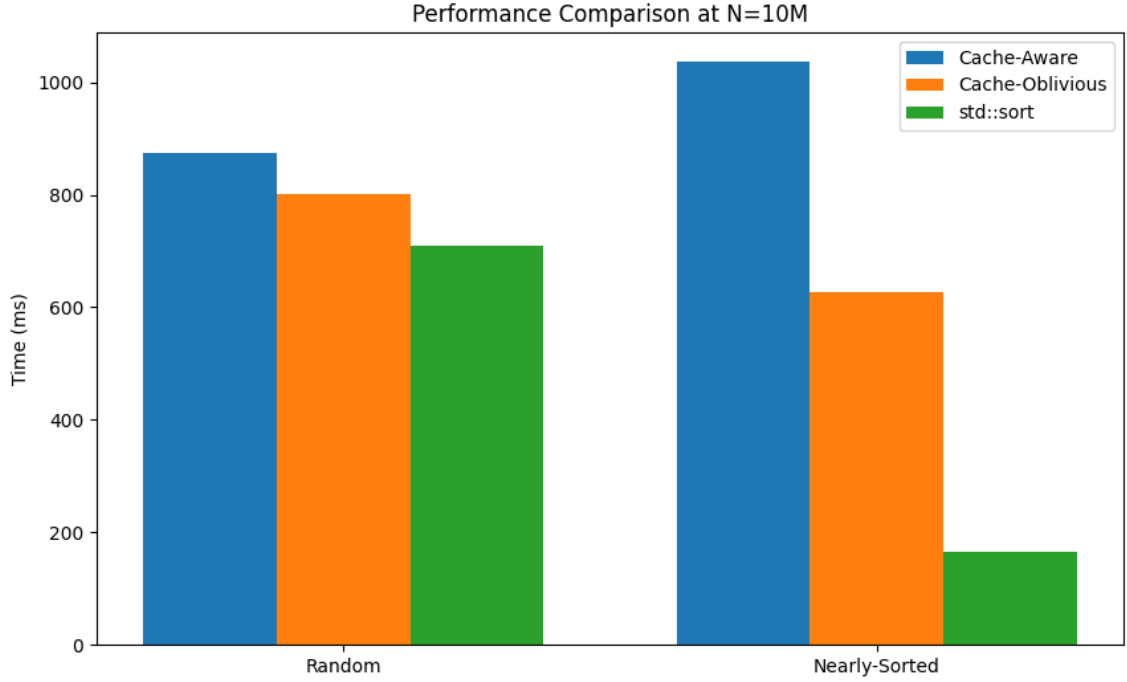


Figure 3: Comparison Table

benchmarks, a $10\times$ increase in input size N (e.g., from 10^6 to 10^7) yields approximately a $10.6\times$ increase in execution time. This ratio is consistent with the theoretical $\Theta(N/B \cdot \log_{M/B}(N/B))$ I/O complexity, where the $\log_{M/B}$ factor behaves as a near-constant overhead as N grows, confirming that our implementations effectively maintain cache-efficient scaling across orders of magnitude. In nearly-sorted conditions, `std::sort` shows a dramatic performance lead, confirming its ability to adapt to input entropy, whereas our Merge Sort variants demonstrate a more rigid, consistent performance floor.

6 Hardware Bottlenecks

We have mitigated branch misprediction by optimizing comparison logic. This shifts the bottleneck from CPU control-flow logic to memory bandwidth limits. Future work involves further optimizing the Cache-Aware k -way merge to reduce the constant factors associated with Winner Tree updates, which appear to contribute to its slightly higher latency compared to the Cache-Oblivious approach in high- N random scenarios.

7 Compiler Flags

All implementations must be compiled with `-O3 -march=native -flto -funroll-loops`.

Appendix: Source Code

For full build instructions, please refer to:

https://github.com/ashdane/IAE-Course_Project